

Homework #5

[ECE30021] Operating Systems

Mission



■ Solve problem 1, 2, and 3

- Solve the problems on Ubuntu Linux (VirtualBox or UTM) using vim and gcc.
- Problem 1: discussion within study group and using AI are allowed
- Problems 2 and 3: for individual students (discussion and use of Internet/AI are prohibited)

■ Submission

- Submit a .tgz file containing hw5_1.c, hw5_2.c and hw5_3.c on LMS.
“tar cvfz hw5_<student_id>.tgz hw5_1.c hw5_2.c hw5_3.c”
(Do not copy&past but type the above command manually!)
- After compression, please check the .tgz file by decompressing in an empty directory.
“tar xvfz hw5_<student_id>.tgz”

■ Due date: PM 11:00:00, June 2nd

Honor Code Guidelines

■ “과제”

- 과제는 교과과정의 내용을 소화하여 실질적인 활용 능력을 갖추기 위한 교육활동이다. 학생은 모든 과제를 정직하고 성실하게 수행함으로써 과제에 의도된 지식과 기술을 얻기 위해 최선을 다해야 한다.
- 제출된 과제물은 성적 평가에 반영되므로 공식적으로 허용되지 않은 자료나 도움을 획득, 활용, 요구, 제공하는 것을 포함하여 평가의 공정성에 영향을 미치는 모든 형태의 부정행위는 단호히 거부해야 한다.
- 수업 내용, 공지된 지식 및 정보, 또는 과제의 요구를 이해하기 위하여 동료의 도움을 받는 것은 부정행위에 포함되지 않는다. 그러나, 과제를 해결하기 위한 모든 과정은 반드시 스스로의 힘으로 수행해야 한다.
- 담당교수가 명시적으로 허락한 경우를 제외하고 다른 사람, 또는 AI가 작성하였거나 인터넷 등에서 획득한 과제물, 또는 프로그램 코드의 일부, 또는 전체를 이용하는 것은 부정행위에 해당한다.
- 자신의 과제물을 타인에게 보여주거나 빌려주는 것은 공정한 평가를 방해하고, 해당 학생의 학업 성취를 저해하는 부정행위에 해당한다.
- 팀 과제가 아닌 경우 두 명 이상이 함께 과제를 수행하여 이를 개별적으로 제출하는 것은 부정행위에 해당한다.
- 스스로 많은 노력을 한 후에도 버그나 문제점을 파악하지 못하여 동료의 도움을 받는 경우도 단순한 문법적 오류에 그쳐야 한다. 과제가 요구하는 design, logic, algorithm의 작성에 있어서 담당교수, TA, tutor 이외에 다른 사람의 도움을 받는 것은 부정행위에 해당한다.
- 서로 다른 학생이 제출한 제출물간 유사도가 통상적으로 발생할 수 있는 정도를 크게 넘어서는 경우, 또는 자신이 제출한 과제물에 대하여 구체적인 설명을 하지 못하는 경우에는 부정행위로 의심받거나 판정될 수 있다.

Problem 0



- Run this command on your VM to install manpages
 - `sudo apt install manpages-posix-dev`
- Search the Internet for the following functions and learn how to use mutex and semaphore
 - Mutex (`pthread_mutex_t`) functions
 - `pthread_mutex_init()`, `pthread_mutex_destroy()`
 - `pthread_mutex_lock()`, `pthread_mutex_unlock()`
 - Semaphore (`sem_t`) functions
 - `sem_open()`, `sem_close()`, `sem_unlink()` // for named semaphores
 - `sem_init()`, `sem_destroy()` // for unnamed semaphores
 - `sem_wait()`, `sem_post()`

Problem 1



■ Mission: peer review of HW#4 problems as a group

- Review the solutions of other members and share comments to each other.
- Each member update her/his solutions individually applying the comments.
 - Name the updated solution to Problem 2 as hw5_1_<student_ID>.c
 - Mark the modified parts by comments as the next page.
 - If you believe your previous solution was perfect, submit it again and put a comment “My previous solution was perfect, so I didn't modify it!” in the first line.
- In hw5_1_<student_ID>.c, write the advices you gave to your group mates as comments. (Hangul comments are okay.)

/* Examples of comments to other members

Comments to other_member_1:

- Line ##: This line is buggy. Fix in this way...
- Line ##: This line can be rewritten more efficiently like ...
- ...

Comments to other_member_2:

- Line ##: Initialize the variable i
- Line ##: Use increase the size of aaaa_array

*/

Problem 1



- Example of updated solution.

```
int main()
{
    int i = 0;

    // previous code
    /*
    <old code>

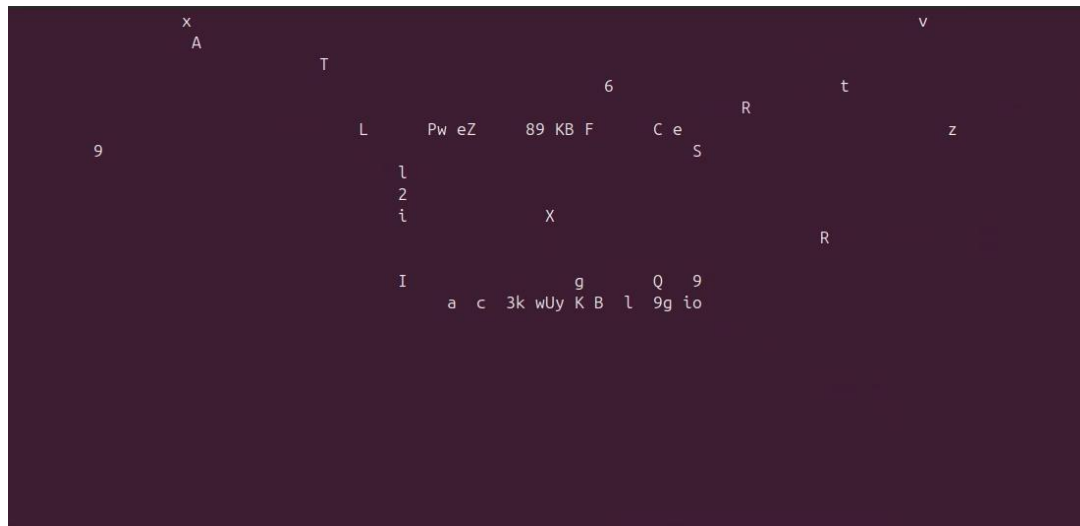
    */

    // updated code
    <new code>

    return 0;
}
```

Problem 2

- Write a multi-threaded program that visualizes a 'critical region' by animation
 - See hw5_2_demo.mp4.
 - Complete hw5_2_problem.c following instructions.
 - If the user presses ESC, display a good bye message and terminate.
 - Random characters moves from a boundary point to the opposite position



Problem 2



■ Key requirements

- At each time, only three characters can enter in the critical region at maximum (use unnamed semaphore)
- Synchronize output instructions (gotoxy(), putchar(), printf(), etc.) not to leave spurious characters on the screen

Hint) Read the synchronization code in hw4_2_problem.c and learn how to determine which sections should be protected by mutual exclusion.

■ Compilation

- `gcc hw5_2.c Console.c -o hw5_2`

Problem 3

- Implement the solution to the bounded buffer problem introduced in chapter 7.
 - See hw5_3_demo.mp4
 - Complete hw5_3_problem.c following instructions.
 - A producer thread inserts messages into the shared buffer
 - A consumer thread removes messages into the shared buffer

```
Message for you: [Every day is a new beginning. Take a deep breath and start again.]

count = 4
I Believe in yourself-you've made it through before, and you will again.
Your story isn't over yet.
You're not alone. I'm here for you.
Every day is a new beginning. Take a deep breath and start again.
```

Problem 3



- The user specifies the interval of the producer and consumer as command line arguments.

- `./hw5_3 <prod_interval> <cons_interval>`

Ex) `./hw5_3 2 2` // the interval of both threads are 2 sec.
// the buffer is in a stable state

Ex) `./hw5_3 2 1` // the producer inserts every 2 seconds
// the consumer deletes every second
// the buffer can be empty

Ex) `./hw5_3 1 2` // the producer inserts every second
// the consumer deletes every 2 seconds
// the buffer can be full

Problem 3



■ Key requirements

- If the bounded buffer solution is correctly implemented, the queue does not display full or empty message.
- The program should not leave spurious characters on the screen.

■ Compilation

- `gcc hw5_3.c StringQueue.c Console.c -o hw5_3`